

TrendPilot: Agentic AI for Instagram Reels

PITB AI Internship — Task 2
Comprehensive Technical Project Report

Autonomous Multi-Agent AI Assistant for End-to-End Instagram Reel Content Generation

Technology Stack

Layer	Technology	Purpose
Orchestration	LangGraph	Stateful multi-agent workflow
LLM	Ollama + gemma3:4b	Local AI inference
Interface	Streamlit	Interactive application UI
Persistence	SQLite	Agent memory and historical content

1. EXECUTIVE SUMMARY

TrendPilot is an autonomous, production-grade multi-agent AI assistant designed to streamline the end-to-end creation of viral Instagram Reels. The system decomposes the content-generation workflow into specialized AI agents while a state-machine orchestration layer coordinates execution and preserves structured intermediate state.

The solution combines LangGraph state-machine orchestration, local Ollama inference using gemma3:4b, an interactive Streamlit user interface, and persistent SQLite storage. This design keeps the application modular and locally executable while providing an extensible foundation for future integrations.

Core Technology Stack

Component	Technology	Responsibility
Agent orchestration	LangGraph	Stateful multi-agent workflow and node transitions
Local LLM	Ollama + gemma3:4b	Local inference for generation and review
UI	Streamlit	Interactive inputs, generation workflow, and history
Persistence	SQLite	Durable agent-memory and generated-content storage
Content format	Markdown	Portable generated artifacts
Runtime	Python	Application, agent, tools, persistence, and tests

2. SYSTEM ARCHITECTURE & WORKFLOW

2.1 High-Level Architecture

User → Streamlit UI → LangGraph State Machine → Trend Idea Generator → Reel Script Writer → Hashtag Generator → Content Reviewer → SQLite Memory + Markdown Artifacts → Streamlit Historical Retrieval

2.2 Node 1 — Trend Idea Generator

The first agent analyzes topic parameters and the selected tone, transforming a broad request into a concrete Reel concept. Responsibilities include topic interpretation, tone alignment, content-angle selection, and passing the concept to downstream script generation.

2.3 Node 2 — Reel Script Writer

The Reel Script Writer converts the concept into a structured 30–60 second Reel script. The output includes timestamped sections, narration/dialogue, visual directions, audio/music cues, a hook, progression, and an appropriate closing call-to-action.

00:00–00:05 Hook
00:05–00:15 Context / Setup
00:15–00:35 Main Value / Story
00:35–00:50 Payoff
00:50–01:00 CTA / Closing

2.4 Node 3 — Hashtag Generator

The Hashtag Generator categorizes Instagram tags into high-volume, niche, and topic-specific buckets. This creates a structured hashtag strategy rather than an undifferentiated list.

2.5 Node 4 — Content Reviewer

The Content Reviewer acts as a senior social media director and evaluates hook strength, pacing, clarity, audience relevance, visual/audio alignment, retention and sharing potential, and overall viral potential.

2.6 Persistence & Memory Layer

The upgraded SQLite database, outputs/agent_memory.db, stores generated content persistently. The full Markdown representation is retained in the full_content field, allowing exact historical reconstruction without regeneration. Streamlit exposes historical generations through st.expander components.

2.7 End-to-End Workflow

User Input → Trend Idea → Script → Hashtags → Review → Full Markdown Content → SQLite + generated_scripts/*.md → Streamlit Current Output + History

3. TECHNICAL IMPLEMENTATION & DIRECTORY STRUCTURE

3.1 Clean Repository Layout

```
TrendPilot/
├── app/
│   ├── agent.py
│   ├── main.py
│   ├── memory.py
│   ├── prompts.py
│   └── tools.py
├── outputs/
│   ├── agent_memory.db
│   ├── generated_scripts/
│   └── screenshots/
│       ├── ollama_running.png
│       ├── demo_output.png
│       └── database_memory.png
├── report/
│   └── final_report.pdf
├── tests/
│   └── test_cases.md
```

3.2 Application Modules

File	Role
app/agent.py	LangGraph agent/state-machine workflow and node execution
app/main.py	Streamlit application entry point and user-facing workflow
app/memory.py	SQLite persistence and historical content retrieval
app/prompts.py	Prompt templates and agent-specific instructions
app/tools.py	LLM/tool integration and prompt invocation utilities

3.3 Output and Evidence Artifacts

The outputs directory contains agent_memory.db and generated_scripts/*.md. The screenshots directory contains ollama_running.png, demo_output.png, and database_memory.png. The tests directory contains test documentation, while report contains the final PDF report.

4. ERROR ANALYSIS & TROUBLESHOOTING LOG

The following issues were encountered during implementation and resolved through changes to execution context, prompt handling, filesystem conventions, runtime configuration, and state isolation.

#	Real-World Issue	Root Cause	Resolution	Result
1	TypeError: 'str' object is not callable	Missing .format() on prompt templates.	Appended proper keyword arguments and formatting in tools.py.	Prompt construction returned the expected formatted string.
2	ModuleNotFoundError: 'app'	PowerShell path context loss during direct script calls.	Used python -m streamlit run app/main.py.	Project-root imports resolved consistently.
3	Unorganized Folder Creation	Singular/plural typo: output/ vs outputs/.	Standardized targets to outputs/generated_scripts/.	Generated files were consolidated.
4	HTTP Read Timed Out (300s limit)	Local hardware token-generation bottlenecks on complex prompts.	Extended HTTP timeouts to 300 seconds and optimized prompt boundaries.	Long-running local inference received a controlled timeout window.
5	Reviewer Context Pollution	Residual error-log text leaked into intermediate state payloads.	Isolated tool inputs and maintained strict execution separation.	Reviewer received clean content context.

4.1 Troubleshooting Principles

1. Treat prompt construction as code. 2. Preserve a stable execution context. 3. Centralize filesystem conventions. 4. Design for local inference latency. 5. Keep state boundaries strict.

5. MULTI-LEVEL TEST CASES EVALUATION

Testing was structured across five complexity levels, progressing from basic Python correctness to distributed asynchronous task processing.

Level	Complexity	Test Focus	Expected Outcome
1	Basic	Python syntax debugging, imports, functions, strings, simple execution	Syntax/runtime errors are detected and corrected
2	Intermediate	Prompt formatting, tool invocation, state propagation, paths, database operations	Components exchange valid data
3	Advanced	LangGraph execution, Ollama inference, Streamlit, persistent memory	Full workflow executes through review and storage
4	Expert	Failures, timeouts, context isolation, repeated generations, historical retrieval	Application remains controlled under realistic stress
5	Distributed / Production-oriented	Celery + Redis queues, retries, worker execution	Long-running workloads can be decomposed into background tasks

5.1 Level 1 — Python Syntax Debugging

Verifies syntax, imports, function invocation, string handling, and basic exception identification.

5.2 Level 2 — Component and Integration Testing

Tests prompt formatting, tool arguments, filesystem targets, SQLite operations, state payload structure, and function boundaries.

5.3 Level 3 — Full Agent Workflow

Validates the complete sequence: User Input → Trend Idea → Script → Hashtags → Review → Persistence → UI Retrieval.

5.4 Level 4 — Reliability and Failure Handling

Focuses on local inference latency, timeout behavior, repeated generation, historical retrieval, reviewer state isolation, and malformed intermediate conditions.

5.5 Level 5 — Distributed Task Processing

Extends the architecture toward asynchronous Celery and Redis execution for long-running AI generation workloads, including worker-based execution, task queues, retries, and non-blocking web interactions.

Streamlit Request → Redis Broker → Celery Worker → AI Generation Task → Persistence → UI Result Retrieval

6. CONCLUSION & DELIVERABLES STATUS

TrendPilot fulfills the defined repository and internship-task requirements as a production-oriented multi-agent AI engineering project.

The completed implementation provides LangGraph orchestration, four specialized AI stages, local Ollama inference with gemma3:4b, a Streamlit interface, persistent SQLite memory, full_content retention, historical retrieval, organized generated-script storage, screenshot evidence, a five-issue troubleshooting record, and a five-level test matrix.

Deliverables Status

Deliverable	Status
Multi-agent LangGraph workflow	Completed
Trend Idea Generator	Completed
Reel Script Writer	Completed
Hashtag Generator	Completed
Content Reviewer	Completed
Ollama gemma3:4b integration	Completed
Streamlit UI	Completed
SQLite persistent memory	Completed
full_content historical storage	Completed
Markdown generated-script storage	Completed
Screenshot evidence	Completed
Troubleshooting documentation	Completed
Multi-level test matrix	Completed
Final technical report	Completed

Final Assessment

The resulting architecture demonstrates core engineering characteristics expected from an agentic AI application: modular agents, explicit orchestration, controlled state propagation, local model integration, persistent memory, user-facing observability, structured outputs, and documented failure handling.

TrendPilot is positioned as a complete internship submission artifact and as a practical foundation for future extensions such as real-time trend ingestion, external social-media APIs, analytics-driven feedback loops, asynchronous worker execution, automated publishing, and model/provider abstraction.